



Project Title: Knowledge Graph for Austrian Company Ownership and Control

Name: Babak Bayani

Mode: ☐ 6 ECTS / ☒ 3 ECTS

Representations

(LO1) Understand and apply Knowledge Graph Embeddings	☐ I showed basic proficiency ☒ exceeded basic proficiency
I implemented a TransE Knowledge Graph Embedding model in PyTorch and trained it on the ownership graph, then evaluated the learned embeddings using a leave-one-out link-prediction experiment with MRR and Hits@K metrics.	Page 5,6
(LO2) Understand and apply logical knowledge in KGs	☐ I showed basic proficiency ☐ exceeded basic proficiency
(LO3) Understand and apply Graph Neural Networks	☐ I showed basic proficiency ☐ exceeded basic proficiency
(LO4) Compare different Knowledge Graph data models from the database, semantic web, machine learning and data science communities.	☐ I showed basic proficiency ☐ exceeded basic proficiency

Systems

(LO5) Design and implement architectures of a Knowledge Graph	☒ I showed basic proficiency ☐ exceeded basic proficiency
I designed the project as a small layered KG system with separate stages for data preparation, graph construction, reasoning, embedding generation, and frontend visualization.	Page 5,7
(LO6) Describe and apply scalable reasoning methods in Knowledge Graphs	☐ I showed basic proficiency ☒ exceeded basic proficiency



I implemented multi-hop rule-based reasoning for direct and indirect control, including configurable control thresholds, maximum reasoning depth, cycle protection, reasoning paths, and controller queries.	Page 6
(LO7) Apply a system to create a Knowledge Graph	☐ I showed basic proficiency ☒ exceeded basic proficiency
I built a complete KG creation pipeline starting from a curated Austrian ownership dataset. The pipeline validates and normalizes the input data, constructs graph nodes and ownership edges, generates metadata and quality statistics, derives control relations, creates embeddings, and validates consistency between the resulting artifacts.	Page 4, 5, 7
(LO8) Apply a system to evolve a Knowledge Graph	☐ I showed basic proficiency ☐ exceeded basic proficiency

Applications

(LO9) Describe and design real-world applications of Knowledge Graphs	☒ I showed basic proficiency ☐ exceeded basic proficiency
I applied the Knowledge Graph approach to Austrian company ownership and control, where ownership chains can be difficult to understand in tabular form. The prototype provides a practical way to explore ownership structures and identify direct and indirect control.	Page 4, 7
(LO10) Describe financial Knowledge Graph applications	☒ I showed basic proficiency ☐ exceeded basic proficiency
I applied Knowledge Graph to a financial domain by representing company ownership and control relationships. The resulting structure can support analysis of corporate ownership, investment relationships, and control structures.	Page 4, 7
(LO11) Apply a system to provide services through a Knowledge Graph	☐ I showed basic proficiency ☐ exceeded basic proficiency



(LO12) Describe the connections between Knowledge Graphs (KGs), Machine Learning (ML) and Artificial Intelligence (AI)	☒ I showed basic proficiency ☐ exceeded basic proficiency
I combined Knowledge Graph reasoning with a machine-learning-based TransE embedding model, providing a practical example of how explicit graph structure and learned vector representations can complement each other.	Page 5, 6

Additional Information

How many hours did you spend on your mini-project ?	100-110 hours
--	---------------

* please exclude any hours you spent on parts reused from other courses

How many hours did you spend on your portfolio document preparation (this PDF) ?	12 hours
---	----------

* please exclude any hours you spent on parts reused from other courses

Declaration

I have marked all parts generated by Generative AI (e.g., ChatGPT) or otherwise copied from other sources and given any prompt I used either in a footnote or in an appendix making clear which parts are generated by which prompts or similar.	☒ I confirm this
--	--

Please indicate if you have used generative AI parts of the mini-project	☒ I used it for roughly 25–35% of the mini-project
Generative AI was used primarily as a development assistant for debugging, documentation, and report drafting. All implementation decisions, testing, integration, and final verification were performed by me.	

Please indicate if you have used generative AI parts of this document	☒ I used it for roughly 30% of this document
Generative AI was used to assist with drafting, formatting and refining the document. The technical content, experimental results, and final revisions were done by me.	



1. Scenario and Motivation

Company ownership is naturally represented as a network: one investor can own a company, that company can own another company, and control can continue across several levels. In a tabular dataset, these relationships are harder to follow. The project therefore uses a Knowledge Graph to represent Austrian ownership structures and infer direct and indirect control.

The goal was to build a prototype covering the full workflow: data preparation, graph construction, reasoning, embeddings, validation, and browser-based exploration.

Three tasks were considered scope of this project: represent ownership relationships, infer direct and indirect control, and provide an interface for exploring the results. Python, CSV/JSON, Angular, and vis-network were utilized; no graph database or backend was introduced. (LO5, LO9, LO10)

2. Knowledge Graph Construction

2.1 Data Sources and Curation

The dataset is a curated subset of Austrian company and ownership information. I used the Austrian Business Register High Value Dataset on data.gv.at, JustizOnline Firmenbuch, FinAPU Firmenbuch, and public ownership information from organisations and companies such as ÖBAG, Österreichische Post AG, OMV AG, VERBUND AG, and Telekom Austria AG. The dataset is not a complete or live extraction of the Austrian Firmenbuch; it was prepared as a small educational dataset for the project.

The raw structure was simplified to three fields: owner, target, percentage. This was enough for the intended graph and reasoning tasks while keeping the input transparent and easy to inspect. The cleaning script checks required fields, empty names, percentage range, self-ownership, whitespace, and duplicates. The resulting quality report contains 9 valid records, 0 invalid records, 0 duplicates removed, 10 unique entities, and 9 ownership relations. (LO7)

Measure	Result
Input rows	9
Valid rows	9
Invalid rows	0
Duplicates removed	0
Unique entities	10
Ownership relations	9

2.2 Graph Schema and Pipeline

Each unique entity becomes a node and each ownership statement becomes a directed (OWNS) edge from owner to target. The ownership percentage is stored as



an edge property. Direct and indirect control are kept separate from asserted ownership: they are generated later by the reasoning stage.

The pipeline is implemented as separate Python scripts:

raw CSV -> cleaning -> cleaned JSON -> graph construction -> reasoning -> TransE embeddings -> validation -> frontend export.

Each stage was Separated to make debugging easier and allow each output to be checked independently. The validation step previously detected an inconsistency between the current graph entities and an older embedding file; regenerating the embeddings fixed the mismatch and the final pipeline validation passed. (LO7)

The system has four simple layers: data, knowledge graph, reasoning/machine learning, and application. JSON was chosen instead of Neo4j because the graph is very small and the main goal was to demonstrate the Knowledge Graph workflow rather than database infrastructure. (LO5)

3. Knowledge Graph Embeddings with TransE

In the project I use rule-based reasoning for transparent control inference, but I also implemented a Knowledge Graph Embedding model to explore a machine-learning representation of the same graph. Embeddings represent entities and relations as vectors and can be used to score or rank possible links.

3.1 TransE Implementation and Evaluation

I implemented TransE directly in PyTorch. For a valid triple, the model learns an approximate relation $h + r \approx t$ and uses a distance-based score. Positive ownership triples are combined with negative samples created by corrupting the head or tail. The model uses 32-dimensional embeddings, 1000 training epochs, learning rate of 0.01, margin of 1.0, and a seed of 42. The trained entity and relation vectors are exported to embeddings.json.

Training loss decreased substantially, however small graph makes the training loss alone into a weak measure for quality. I therefore implemented leave-one-out link prediction: each of the 9 ownership triples was held out in turn, the model was retrained on the remaining triples, and candidate target entities were ranked.

Metric	Result
MRR	0.2059
Hits@1	0.0000
Hits@3	0.2222
Hits@10	1.0000

The embedding results are modest and should not be interpreted as strong predictive performance. There are only 10 entities, so the candidate space is very small; Hits@10 = 1.0 is therefore not particularly informative, and Hits@1 = 0 means the correct target was never ranked first. The main achievement for LO1 is the



complete practical workflow: graph-to-triples conversion, negative sampling, TransE training, embedding export, and held-out evaluation with MRR and Hits@K. (LO1)

The embedding and reasoning approaches are complementary. The rule-based system uses explicit domain assumptions and stores an explainable path, while TransE learns numerical representations without understanding the meaning of control. Keeping both approaches in the project helped connect symbolic and machine-learning aspects of Knowledge Graphs. (LO1, LO12)

4. Reasoning for Ownership and Control

The reasoning layer distinguishes asserted ownership from inferred control. For this project an ownership percentage of at least 50% is treated as direct control. The threshold is configurable and the current maximum reasoning depth is 10.

4.1 Direct and Indirect Control

For example, Österreichische Beteiligungs AG owns 52.85% of Österreichische Post AG, so the system derives DIRECT_CONTROL. Its 31.5% stake in OMV AG remains an OWNS relationship and does not produce direct control under the 50% rule.

Indirect control is derived by traversing the direct-control graph. A chain such as Republic of Austria -> Österreichische Beteiligungs AG -> Österreichische Post AG -> bank99 AG produces an indirect control relation between the Republic of Austria and bank99 AG. Each inferred relation stores its path and depth, making the result explainable instead of returning only a Boolean answer.

4.2 Protection and Verification

The traversal has two safeguards: a maximum depth and cycle protection. Before extending a path, the algorithm checks whether the next entity already occurs in the path. This prevents repeated traversal through cycles and gives predictable bounds on the search.

A controller query was also implemented. For bank99 AG, the current result identifies Österreichische Post AG as a direct controller and Österreichische Beteiligungs AG and the Republic of Austria as indirect controllers.

The reasoning implementation was tested independently with synthetic A-B-C-D graphs. The tests cover direct control, rejection below the threshold, multi-hop inference, reasoning paths and depth, controller queries, cycle protection, and the maximum depth. All tests were passed successfully. The current Austrian graph produced 5 direct-control and 3 indirect-control relations. (LO6)

5. Application and System Architecture

The Angular frontend uses the generated JSON rather than performing reasoning itself. This keeps the Python processing pipeline independent from the user interface. The application provides a company list with search, a company detail view, and an interactive graph using vis-network.

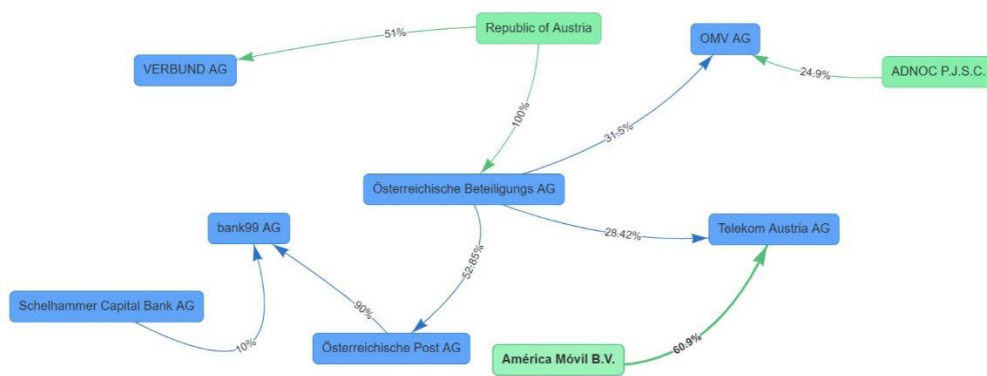


Figure 1: Interactive visualization of the Austrian company ownership Knowledge Graph.

The detail view illustrates ownership and control information, including direct and indirect controllers. For example, bank99 AG can be inspected along with the inferred controller relationships. The graph view focuses on ownership edges and shows ownership percentages, which makes multi-step structures easier to understand than isolated table rows.

This provides a small real-world use case for Knowledge Graphs in financial and corporate-structure analysis: a user can move from a company to its owners and subsidiaries and inspect control relationships derived from the graph. (LO5, LO9, LO10)

6. Results and Evaluation

The final prototype successfully connected the data, graph, reasoning, embedding, validation, and frontend stages. The main quantitative results are summarized below.

Component	Result
Knowledge graph	10 nodes, 9 OWNS edges
Data validation	9 valid, 0 invalid, 0 duplicates
Control reasoning	5 direct, 3 indirect
Control settings	50% threshold, depth 10
TransE	32 dimensions, 1000 epochs
Embedding evaluation	MRR: 0.2059 Hits@1: 0 Hits@3: 0.2222 Hits@10: 1.0
Reasoning tests	All passed
Pipeline validation	All checks passed

The pipeline validation confirms that cleaned relations exist in the graph, graph edges reference valid nodes, metadata is consistent, control relations reference valid nodes, and every graph entity has a 32-dimensional embedding. Because the validator previously caught a graph/embedding mismatch, it also served as a practical consistency check between pipeline stages rather than only a final status message.(LO7)



Overall, the most important outcome is the integration of several Knowledge Graph components into one reproducible system. The embedding metrics are intentionally treated as exploratory because the graph is too small for strong statistical conclusions. The reasoning implementation and its automated verification provide clear evidence of deterministic behaviour. (LO1, LO6, LO7)

7. Limitations and Future Improvements

The dataset contains only 9 ownership relations and 10 entities, so it cannot represent the Austrian company landscape comprehensively. A larger dataset would be needed to make reasoning and embedding evaluation more meaningful. The current control definition is also simplified: the 50% rule does not cover voting agreements, contractual rights, or other forms of effective controls.

JSON storage, no backend, and a simple Angular interface are used for this project, but a larger system could use a graph database, an API, richer control rules, and much more advanced graph filtering. The embedding evaluation could also be extended with a larger dataset and comparisons to additional embedding models.

8. Conclusion and Learning Outcome Reflection

The project achieved its main goal of building a small and complete Knowledge Graph for Austrian company ownership and control. Starting from curated public source data, validation and graph were implemented, direct and indirect control were extracted through multi-hop reasoning, TransE embedding model was trained and evaluated, generated artifacts were validated, and the results were demonstrated through an Angular application.

For LO7, the project demonstrates the full KG creation pipeline from curated source data through validation, graph generation, metadata, reasoning, embeddings, and final consistency checks. For LO6, the reasoning system goes beyond a simple control label by supporting multi-hop paths, configurable depth, cycle protection, controller queries, and automated tests. For LO1, I implemented and evaluated TransE as well.

The remaining outcomes are demonstrated as such: LO5 through the layered architecture, LO9 through the ownership-analysis application, LO10 through the financial domain, and LO12 through the combination of symbolic reasoning and machine-learned embeddings.